





Python

02.12.2025

RECAP - import

Importowanie bibliotek do kodu:

```
import biblioteka
```

Importowanie biblioteki jako alias:

```
import biblioteka as alias
```

RECAP - Komendy git

git init	inicjalizuje nowe repo
git clone	kopiuje istniejące repo
git pull	pobiera zmiany
git push	wysyła zmiany
git add nazwa	dodaje plik
git commit -a	zatwierdza wszystkie zmiany
git commit -m "msg"	zatwierdza zmiany z wiadomością bez edytora
git status	wyświetla stan repo
git remote add nazwa url	dodaje nowe repo

<https://github.com/DeepDive-UG/Python-ST-2025>

RECAP - Jak działa *for*

Pętla *for* (dla każdego) używana jest do wymieniania elementów z list, krotek, słowników, generatorów, itp.

RECAP - Jak działa `for`

```
lista = ["element1", "element2", "element3"]  
for element in lista:  
    print(element)
```

Dla każdego elementu "element" z listy "lista":

element = "element1"

Wyświetla: element1

element = "element2"

Wyświetla: element2

element = "element3"

Wyświetla: element3

RECAP - Macierze i ich mnożenie

$$3 \cdot 0 + 1 \cdot 2 + 0 \cdot 0 = 2$$

$$\begin{bmatrix} 3 & 1 & 0 \\ 0 & 2 & 1 \end{bmatrix} \times \begin{bmatrix} 0 & 4 \\ 2 & 2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & \\ & \end{bmatrix}$$

$$2 \times 3$$

$$3 \times 2$$

$$2 \times 2$$

RECAP - Transponowanie macierzy

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ d & e & f \end{bmatrix}_{2 \times 3} \quad \mathbf{A}^T = \begin{bmatrix} a & d \\ b & e \\ c & f \end{bmatrix}_{3 \times 2}$$

Punkt startu programu

Punktem startowym programu nazywamy konstrukt:

```
if __name__ == "__main__":  
    zrób coś :D
```

To spowoduje, że napisane pod tym konstruktem komendy zostaną wywołane tylko jeśli uruchomiony będzie plik macierzysty.

Importowanie skryptu nie spowoduje wykonania kodu spod tego konstruktu, bo parametr `__name__` nie będzie `"__main__"`.

Przykład:

Skrypt1.py:

```
def helloworld():  
    print("Hello World!")
```

```
if __name__ == "__main__":  
    helloworld()
```

Skrypt2.py:

```
from Skrypt1 import helloworld  
  
helloworld()
```

Czym jest klasa?

Klasa jest to struktura zawierająca własne parametry i metody. Definiuje się ją za pomocą słowa kluczowego *class*.

```
class MojaKlasa:  
    nazwa = "kurs pytonga"
```

```
print(MojaKlasa)
```

```
print(MojaKlasa.nazwa)
```

Klasa vs Obiekt

Klasa jest zbiorem parametrów i metod pod jedną wiadomą nazwą. Klasy mogą być odwzorowane wielokrotnie pod postacią **obiektów**.

Obiekt jest odwzorowaniem **klasy**. Każdy **obiekt** może różnić się parametrami od innych, nie zmieniając przy okazji parametrów początkowych **klasy**, z której powstał.

Przykład

```
class MojKurs:  
    nazwa = "Kurs pytonga"  
    wielkosc = 14  
    wolne_miejsca = 0
```

```
kurs1 = MojKurs()  
kurs2 = MojKurs()
```

```
kurs2.wielkosc = 28  
kurs2.wolne_miejsca = 14
```

```
print(kurs1.wielkosc)  
print(kurs2.wielkosc)
```

Metody klas

Klasy w porównaniu do **funkcji** mogą zawierać zbiór “podfunkcji” dla swojego działania, nazywanych metodami. Możemy definiować metody **klasy** definiując nowe **funkcje** pod definicją **klasy**.

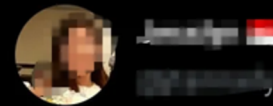
```
class MojaKlasa:  
    def metoda1():  
        print("zrób coś")  
    def metoda2():  
        print("zrób coś innego")
```

Specjalne metody klas - `__init__`

Jeśli chcemy wprowadzić konkretne wartości parametrów dla klasy przy definiowaniu **obiektów** musimy odpowiednio **zainicjować** klasę. Do tego używa się specjalnej metody `__init__`.

```
class MojaKlasa:  
    def __init__(self, param1, param2):  
        self.param1 = param1  
        self.param2 = param2
```

```
obj1 = MojaKlasa("parametr1", "parametr2")  
print(obj1.param1)
```



British Python devs be like "that's a constructor, `__init__`?"

2:08 AM · 11 Feb 21 · [Twitter Web App](#)

360 Retweets 24 Quote Tweets 3,260 Likes

Czym w klasach jest `self`

`self` odwołuje się bezpośrednio do nazwy obiektu lub klasy, tzn. jeśli mamy inicjację klasy za pomocą `__init__`, parametr `self` odwołuje się za nas do nazwy naszej klasy.

Inaczej: Jeśli nasza klasa ma nazwę `MojaKlasa`, parametr `self` będzie przyjmował nazwę `MojaKlasa` w różnych metodach klasy.

Specjalne metody klasy - `__str__`

Metoda `__str__` pozwala nam na wypisanie czegoś przez `print` jeśli podamy tylko `obiekt` albo `klasę` bez definicji parametru który chcemy wyciągnąć.

```
class MojaKlasa:
    def __init__(self, param1, param2):
        self.param1 = param1
        self.param2 = param2
    def __str__(self):
        return f"Klasa MojaKlasa o parametrach {self.param1 = }, {self.param2 = }"

obj1 = MojaKlasa("parametr1", "parametr2")
print(obj1)
```

Kontrola przepływu programu

Jeśli wiemy, że wprowadzone do programu wartości mogłyby spowodować błąd, czy da się coś z tym zrobić, albo zwrócić użytkownikowi komunikat, który dokładnie powie co się stało?

Oczywiście że tak!

Kontrola przepływu programu

W tym celu wykorzystuje się blok *try...except*! Jest to konstrukt, który pozwala zdefiniować próbę wykonania programu (*try*), oraz jeśli coś pójdzie nie tak, pozwoli na poprawienie błędu lub przekierowanie kursu programu w celu jego bezpiecznego zakończenia (*except*).

Przyjrzyjmy się temu bliżej

try:

```
x = input("Wprowadź liczbę całkowitą: ")
```

```
x_num = int(x)
```

```
print(f"Liczba Całkowita: {x_num}")
```

```
# Pobierze od użytkownika tylko liczby całkowite, inaczej zwróci  
ValueError
```

except ValueError:

```
print("Wprowadzona wartość nie jest liczbą całkowitą!")
```

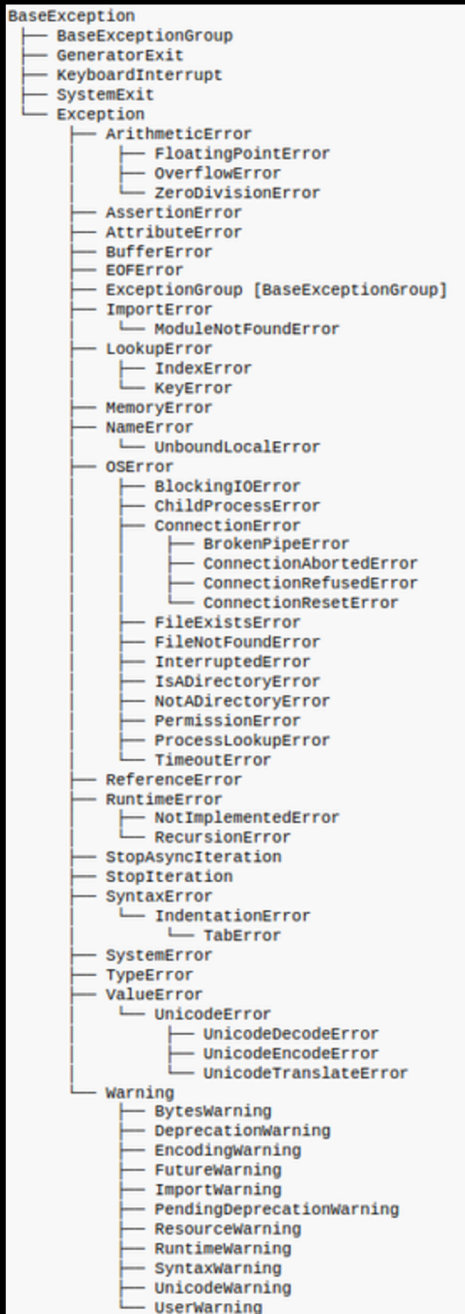
```
print("Program zostanie wyłączony!")
```

```
exit(-1) # Wyłącza program z kodem -1 (błąd, brak sukcesu, itp.)
```

Typy błędów

Istnieje wiele typów błędów wbudowanych w języku Python, ich definicje i przyczyny znajdziecie w dokumentacji języka:

<https://docs.python.org/3/library/exceptions.html>



Wywoływanie błędów ręcznie

Python umożliwia nam również ręczne wywołanie błędu poprzez wykonanie komendy *raise* (wznieść).

Pozwoli nam to wyłapanie potencjalnie niebezpiecznych lub niepożądanych działań programu podczas jego działania i na przykład terminowanie go z podaniem komunikatu dla użytkownika.

Wywoływanie błędów ręcznie

Przykład:

```
x = 10
```

```
y = 0
```

```
if y == 0:
```

```
    raise ZeroDivisionError("Nie można dzielić przez zero!")
```

```
else:
```

```
    print(x/y)
```

Definiowanie własnych błędów

Możemy też w prosty sposób zdefiniować własny typ błędu, jeżeli spotykamy się z problemem spoza zakresu błędów wbudowanych. Są to dwie linijki kodu wyglądające następująco:

```
class WlasnyBlad(BaseException):  
    pass
```


Kontrola przepływu programu - *finally*

Kontrola przepływu programu - *finally*

Do bloku *try...except* można również dodać klauzulę *finally*, która dosłownie oznacza “ostatecznie”. Kod napisany pod nią zostanie wykonany zawsze, nawet jeśli kod spod bloku *try* został wykonany poprawnie.

Kontrola przepływu programu - *finally*

Ponadto, jeśli w bloku try wystąpił błąd który nie jest obsługowany blokiem *except*, błąd ten zostanie wywołany ponownie na końcu kodu w bloku *finally*.

Kontrola przepływu programu - *finally*

Użycie tej klauzuli ogranicza się na dobrą sprawę jako 'clean-up' kodu wykonywanego w blokach *try...except* przy większym ich skomplikowaniu aby zwolnić pamięć zajmowaną przez np. zmienne testowe czy inne dane nie potrzebne do dalszego działania programu.

Kontrola przepływu programu - *finally*

```
x = input("Wprowadź dzielną: ")
y = input("Wprowadź dzielnik: ")
try:
    if "." in x or "." in y:
        raise ValueError
    else:
        x_num = int(x)
        y_num = int(y)

        if y_num == 0:
            raise ZeroDivisionError(f"Podany dzielnik {y = } jest równy zero, nie można kontynuować")
        else:
            print(f"Wynik ilorazu to {x_num / y_num = }")
except ValueError:
    print("Jedna z wprowadzonych danych nie jest liczbą całkowitą. Sprawdź i popraw:\n",
          f"{x = }\n",
          f"{y = }")
    exit(-1)
finally:
    print("Wykonywanie finally")
```

Komenda `assert`

`assert` jest instrukcją w języku Python, która służy do sprawdzania warunków. Jeśli warunek jest fałszywy, `assert` podnosi wyjątek `AssertionError`.

Komenda `assert`

Przykład:

```
def divide(a, b):  
    assert b != 0, "Error: division by zero"  
    return a / b
```

```
result = divide(10, 2)  
print("Result:", result)    # Output: Result: 5.0
```

```
result = divide(10, 0)    # AssertionError: Error: division by zero
```

Moduł unittest

unittest to moduł wbudowany w język Python, który umożliwia pisanie i uruchamianie testów jednostkowych. Testy jednostkowe są narzędziem programistycznym, które pomaga w weryfikacji poprawności poszczególnych fragmentów kodu, tj. jednostek, takich jak **funkcje** czy **klasy**.

Moduł unittest

```
import unittest
```

```
def add(a, b):  
    return a + b
```

```
class TestAddFunction(unittest.TestCase):  
    def test_add_positive_numbers(self):  
        self.assertEqual(add(2, 3), 5)
```

```
    def test_add_negative_numbers(self):  
        self.assertEqual(add(-2, -3), -5)
```

```
if __name__ == '__main__':  
    unittest.main()
```

